



UNIVERSIDADE FEDERAL DE ALAGOAS

Instituto de Computação

Curso de Engenharia da Computação

Larissa Ferreira Dias de Souza
Otávio Joshua Costa Brandão Menezes
Davi Celestino dos Santos Barbosa
Renato Coca Terrazas Freire
João Victor Tenório Venancio

JOGO DA FORÇA COMPETITIVO MULTIJOGADOR COM SOCKETS TCP

Maceió – AL

2026

Larissa Ferreira Dias de Souza
Otávio Joshua Costa Brandão Menezes
Davi Celestino dos Santos Barbosa
Renato Coca Terrazas Freire
João Victor Tenório Venancio

JOGO DA FORÇA COMPETITIVO MULTIJOGADOR COM SOCKETS TCP

Relatório de projeto apresentado à disciplina de Redes de Computadores do curso de Engenharia da Computação da Universidade Federal de Alagoas, como requisito parcial para aprovação na disciplina.

Docente: Prof. Dr. Almir Pereira Guimarães

Universidade Federal de Alagoas – UFAL
Instituto de Computação
Maceió – AL
2026

Sumário

1	ENUNCIADO DO PROJETO	3
2	DESCRIÇÃO GERAL DA APLICAÇÃO	4
3	ARQUITETURA CLIENTE-SERVIDOR	5
4	PROTOCOLO DE COMUNICAÇÃO IMPLEMENTADO	6
4.1	Tipos de mensagens	6
5	FUNCIONALIDADES DESENVOLVIDAS	8
6	USO DE SOCKETS E THREADS	9
7	DIFICULDADES ENCONTRADAS	10
8	POSSÍVEIS MELHORIAS FUTURAS	11
9	INSTRUÇÕES DE EXECUÇÃO	12
9.1	Execução local	12
9.2	Execução em rede local	12
10	CONCLUSÃO	13
11	REFERÊNCIAS BIBLIOGRÁFICAS	14
A	CÓDIGO-FONTE	15
A.1	Cliente	15
A.2	Estado local do cliente	19
A.3	Renderização da interface	21
A.4	Servidor	24
A.5	Estado global do jogo	32
A.6	Gerenciador de palavras	39
A.7	Protocolo de comunicação	40

1 Enunciado do projeto

O projeto desenvolvido consiste em uma aplicação distribuída para um jogo da forca competitivo multijogador. A aplicação foi implementada em Python e utiliza comunicação em rede por meio de sockets TCP, conforme os requisitos definidos para a disciplina de Redes de Computadores.

O objetivo principal foi construir uma aplicação cliente-servidor capaz de permitir que múltiplos jogadores participem de uma mesma partida em tempo real. O servidor é responsável por receber conexões, controlar o estado global do jogo, processar palpites, atualizar pontuações, eliminar jogadores sem tentativas e transmitir as atualizações para todos os clientes conectados.

A aplicação foi projetada sem o uso de WebSockets ou bibliotecas externas de comunicação de alto nível. Dessa forma, a troca de dados foi feita diretamente com primitivas de socket, utilizando funções como `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `sendall()` e `recv()`.

2 Descrição geral da aplicação

A aplicação implementada é um jogo da forca competitivo para terminal. Cada jogador executa um cliente Python e se conecta a um servidor central. Após a conexão, o jogador informa seu nome e aguarda o início da partida.

A partida começa quando pelo menos dois jogadores estão conectados. O limite máximo definido para a aplicação é de três clientes simultâneos. Durante o jogo, os jogadores podem enviar palpites de letra ou tentar adivinhar a palavra completa. O servidor valida cada tentativa, atualiza o estado da palavra, altera a pontuação dos jogadores e informa todos os clientes sobre o novo estado da partida.

Cada jogador inicia com seis tentativas. Ao errar, perde uma tentativa. Quando um jogador zera suas tentativas, ele passa a atuar como espectador, acompanhando a partida sem poder enviar novos palpites. O jogo termina quando todos os jogadores são eliminados, quando resta apenas um jogador ativo ou quando o banco de palavras é esgotado.

3 Arquitetura cliente-servidor

A aplicação segue o modelo cliente-servidor. O servidor atua como ponto central de controle da partida, enquanto os clientes são responsáveis pela interação com os usuários.

No lado do servidor, existe uma thread principal que escuta novas conexões TCP. A cada novo cliente aceito, o servidor cria uma thread dedicada para lidar com a comunicação daquele jogador. Essa abordagem permite que vários clientes sejam atendidos simultaneamente, sem que a comunicação com um jogador bloqueie o funcionamento dos demais.

No lado do cliente, também há separação de responsabilidades. A thread principal lê os comandos digitados pelo jogador e envia mensagens para o servidor. Ao mesmo tempo, uma thread secundária fica responsável por receber mensagens vindas do servidor e atualizar a interface textual do jogo.

```
+---+
|   |
| 0  |
| /|\ |
|   |
+---+

=====

-- PLACAR --
Joshua ◀ você
Pontos: 10 | Tentativas: 2
Davi
Pontos: 8  | Tentativas: 6
Tenorio
Pontos: 4  | Tentativas: 6

Categoria: Animais
Palavra: C O E _ _ O

Letras erradas: K Z T X

Sua vez - digite uma letra ou a palavra completa:
> █
```

Figura 1 – Interface textual do cliente durante uma partida

4 Protocolo de comunicação implementado

A comunicação entre cliente e servidor foi feita por meio de um protocolo de aplicação próprio. As mensagens são codificadas em JSON e transmitidas sobre TCP. Cada mensagem é finalizada com o caractere `\n`, usado como delimitador.

Esse delimitador é necessário porque o TCP trabalha como um fluxo contínuo de bytes. Isso significa que uma mensagem enviada pelo emissor pode chegar fragmentada, ou várias mensagens podem chegar juntas em uma mesma chamada de `recv()`. Para resolver esse problema, foi implementado um mecanismo de enquadramento, também chamado de *framing*.

A função `send_msg()` serializa a mensagem em JSON, adiciona o delimitador de fim de mensagem e envia os bytes pelo socket. A função `recv_msgs()` mantém um buffer de recepção, acumula os fragmentos recebidos e só retorna mensagens completas quando encontra o delimitador `\n`.

4.1 Tipos de mensagens

Mensagem	Direção	Descrição
JOIN	Cliente → Servidor	Enviada quando o cliente entra no jogo, contendo o nome do jogador.
WELCOME	Servidor → Cliente	Confirma a entrada do jogador e informa seu identificador único.
WAITING	Servidor → Cliente	Informa que o servidor ainda aguarda jogadores suficientes para iniciar.
GAME_START	Servidor → Cliente	Indica o início de uma rodada e informa a categoria e o tamanho da palavra.
STATE_UPDATE	Servidor → Cliente	Atualiza o estado da partida, incluindo palavra revelada, jogadores, pontuação e tentativas.
GUESS_LETTER	Cliente → Servidor	Envia uma tentativa de letra.
GUESS_WORD	Cliente → Servidor	Envia uma tentativa de palavra completa.
CORRECT_GUESS	Servidor → Cliente	Informa que um palpite foi correto.

Mensagem	Direção	Descrição
WRONG_GUESS	Servidor → Cliente	Informa que um palpite foi incorreto.
PLAYER_OUT	Servidor → Cliente	Informa que um jogador foi eliminado ou desconectado.
GAME_OVER	Servidor → Cliente	Finaliza a partida e envia vencedor, palavra e placar final.
READY	Cliente → Servidor	Indica que o jogador deseja iniciar uma nova partida.
ERROR	Servidor → Cliente	Envia mensagens de erro, como servidor lotado ou banco de palavras vazio.

Tabela 1 – Mensagens do protocolo de aplicação

5 Funcionalidades desenvolvidas

Entre as principais funcionalidades implementadas, destacam-se:

- Conexão de clientes ao servidor por meio de sockets TCP;
- Identificação dos jogadores por nome e identificador único;
- Limite de até três jogadores conectados simultaneamente;
- Início automático da partida com no mínimo dois jogadores;
- Sorteio de palavras a partir de um banco local;
- Envio de palpites por letra ou palavra completa;
- Atualização da palavra revelada em tempo real;
- Controle individual de tentativas restantes;
- Sistema de pontuação por acertos;
- Eliminação de jogadores que zeram suas tentativas;
- Modo espectador para jogadores eliminados;
- Transmissão do estado do jogo para todos os clientes;
- Tratamento de desconexões inesperadas;
- Vitória por W.O. quando resta apenas um jogador ativo;
- Possibilidade de iniciar uma nova partida após o fim do jogo.

6 Uso de sockets e threads

O uso de sockets TCP foi essencial para permitir a comunicação entre processos executando em máquinas iguais ou diferentes. O servidor utiliza o endereço `0.0.0.0` para aceitar conexões externas na rede local, enquanto os clientes podem se conectar usando `localhost` em testes locais ou o endereço IP real da máquina servidora em testes via LAN.

O uso de threads foi necessário porque operações como `accept()` e `recv()` são bloqueantes. Se o servidor atendesse todos os clientes em uma única linha de execução, a comunicação com um jogador poderia travar o atendimento dos demais. Com uma thread por conexão, cada cliente pode enviar mensagens independentemente.

No cliente, a thread de recepção também é importante. Enquanto o usuário digita seus palpites na thread principal, outra thread permanece aguardando mensagens do servidor. Isso permite que atualizações da partida sejam recebidas e renderizadas mesmo quando o jogador ainda não digitou uma nova entrada.

7 Dificuldades encontradas

Durante o desenvolvimento, uma das principais dificuldades foi lidar com a natureza do TCP como fluxo de bytes. Inicialmente, poderia parecer que cada envio corresponderia exatamente a uma leitura, mas isso não é garantido. Foi necessário implementar um protocolo com delimitador e buffer para reconstruir corretamente as mensagens JSON recebidas.

Outra dificuldade foi o controle de concorrência no servidor. Como várias threads acessam o mesmo estado global do jogo, foi necessário utilizar `threading.Lock()` para proteger estruturas compartilhadas, como lista de jogadores, sockets conectados, pontuação, tentativas e fase da partida.

Também houve dificuldade no tratamento de desconexões. Um cliente pode fechar o terminal, perder conexão ou derrubar o socket de maneira inesperada. Por isso, o servidor precisou tratar exceções como `ConnectionResetError` e `OSError`, removendo o jogador da partida sem derrubar todo o servidor.

Além disso, foi necessário separar corretamente o estado local do cliente e o estado global do servidor. O servidor é a fonte principal da verdade da partida, enquanto o cliente apenas mantém uma cópia local para renderizar a interface.

8 Possíveis melhorias futuras

Como melhorias futuras, poderiam ser implementados:

- Autenticação de jogadores por usuário e senha;
- Salas independentes para múltiplas partidas simultâneas;
- Ranking persistente salvo em arquivo ou banco de dados;
- Suporte a reconexão de jogador após queda de rede;
- Interface gráfica em vez de apenas terminal;
- Criptografia na comunicação entre cliente e servidor;
- Logs detalhados de eventos da partida;
- Configuração dinâmica de número máximo de jogadores e tentativas.

9 Instruções de execução

A aplicação foi desenvolvida em Python e não depende de bibliotecas externas. Para executar o projeto, é necessário possuir Python 3.8 ou superior instalado. Para acessar e entender com mais detalhes de como rodar o código e outras informações, acesse o link do repositório [aqui](#).

9.1 Execução local

No Linux ou macOS, abra um terminal na raiz do projeto e execute o servidor:

```
1 python3 servidor/game_server.py
```

Em seguida, abra dois ou três terminais adicionais e execute o cliente:

```
1 python3 cliente/client.py
```

No Windows, os comandos equivalentes são:

```
1 python servidor/game_server.py
```

E, em outros terminais:

```
1 python cliente/client.py
```

9.2 Execução em rede local

Para executar em computadores diferentes, o servidor deve ser iniciado em uma máquina da rede. Os clientes devem informar o IP da máquina servidora.

Na máquina servidora, execute:

```
1 python servidor/game_server.py --host 0.0.0.0 --port 5000
```

Nos clientes, execute substituindo o IP pelo endereço real da máquina servidora:

```
1 python cliente/client.py 26.xxx.xx.x --port 5000
```

Caso esteja utilizando VPN local, como Radmin VPN, deve-se usar o IP fornecido pela VPN.

10 Conclusão

O projeto permitiu aplicar, de forma prática, conceitos fundamentais da disciplina de Redes de Computadores. A aplicação utilizou diretamente sockets TCP para estabelecer comunicação entre processos, implementou um protocolo próprio de aplicação com mensagens JSON delimitadas e aplicou threads para viabilizar o atendimento concorrente de múltiplos clientes.

Além do aspecto funcional do jogo, o desenvolvimento evidenciou problemas reais de aplicações distribuídas, como entrega parcial de mensagens, bloqueio em operações de rede, concorrência no acesso ao estado compartilhado e tratamento de desconexões. Dessa forma, o projeto cumpriu o objetivo de demonstrar uma aplicação cliente-servidor funcional utilizando primitivas de rede em Python.

11 Referências bibliográficas

- KUROSE, James F.; ROSS, Keith W. **Redes de Computadores e a Internet: uma abordagem top-down**. 8. ed. São Paulo: Pearson, 2021.
- TANENBAUM, Andrew S.; WETHERALL, David J. **Redes de Computadores**. 5. ed. São Paulo: Pearson, 2011.
- PYTHON SOFTWARE FOUNDATION. **socket** — Low-level networking interface. Disponível em: <https://docs.python.org/3/library/socket.html>. Acesso em: 2026.
- PYTHON SOFTWARE FOUNDATION. **threading** — Thread-based parallelism. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 2026.
- PYTHON SOFTWARE FOUNDATION. **json** — JSON encoder and decoder. Disponível em: <https://docs.python.org/3/library/json.html>. Acesso em: 2026.

A Código-fonte

A.1 Cliente

```
1 import json
2 import socket
3 import threading
4 import sys
5 import argparse
6 import os
7 from pathlib import Path
8
9 PROJECT_ROOT = Path(__file__).resolve().parents[1]
10 if str(PROJECT_ROOT) not in sys.path:
11     sys.path.insert(0, str(PROJECT_ROOT))
12
13 from utils.protocol import send_msg, recv_msgs
14 from local_state import LocalGameState
15 from interface.renderer import render_state, render_waiting, render_game_over
16
17 DEFAULT_HOST = os.environ.get("HANGMAN_SERVER_HOST", "localhost")
18 DEFAULT_PORT = int(os.environ.get("HANGMAN_SERVER_PORT", "5000"))
19 DEFAULT_TIMEOUT = float(os.environ.get("HANGMAN_CONNECT_TIMEOUT", "8"))
20
21
22 def parse_args():
23     parser = argparse.ArgumentParser(description="Cliente do jogo da forca competitivo.")
24     parser.add_argument(
25         "host",
26         nargs="?",
27         default=DEFAULT_HOST,
28         help="IP ou hostname do servidor. Ex: 192.168.1.10",
29     )
30     parser.add_argument(
31         "--port",
32         type=int,
33         default=DEFAULT_PORT,
34         help="Porta TCP do servidor.",
35     )
36     parser.add_argument(
37         "--timeout",
38         type=float,
39         default=DEFAULT_TIMEOUT,
```



```

40     help="Tempo limite, em segundos, para conectar ao servidor.",
41 )
42 return parser.parse_args()
43
44 def recv_loop(sock, state):
45     #comportamento full-duplex da aplicação (envia e recebe simultaneamente).
46     #Buffer acumulador de fragmentos TCP mesma abordagem usada no servidor.
47     #recv_msgs() acumula chunks parciais e só retorna mensagens quando o delimitador
48     '\n' é encontrado,
49     #evitando que JSONs fragmentados sejam entregues incompletos ao LocalGameState.
50     recv_buffer = [""]
51
52     try:
53         while True:
54             #recv_msgs() é bloqueante: a thread fica ociosa até o SO entregar dados.
55             #Retorna lista vazia quando o servidor envia TCP FIN (conexão encerrada).
56             messages = recv_msgs(sock, recv_buffer)
57
58             if not messages:
59                 print("\nConexão encerrada pelo servidor.")
60                 break
61
62             for msg in messages:
63                 msg_type = msg.get("type")
64
65                 #Atualiza a fonte de verdade com a mensagem já montada e validada.
66                 state.update(json.dumps(msg))
67
68                 #Renderiza a tela sempre que o estado mudar exceto mensagens de
69                 #controle que não alteram a tela principal (ex: WELCOME, ERROR).
70                 if msg_type in ("GAME_START", "STATE_UPDATE", "WRONG_GUESS",
71                                "CORRECT_GUESS", "PLAYER_OUT"):
72                     render_state(state)
73
74                 elif msg_type == "WAITING":
75                     payload = msg.get("payload", {})
76                     render_waiting(
77                         payload.get("connected", 0),
78                         payload.get("needed", 2),
79                     )
80
81                 elif msg_type == "GAME_OVER":
82                     payload = msg.get("payload", {})
83                     render_game_over(
84                         winner_name=payload.get("winner_name"),
85                         word=payload.get("word", "?"),
86                         scores=payload.get("scores", []),
87                     )

```

```

86         )
87
88         print("> ", end="", flush=True)
89
90     except (ConnectionResetError, OSError):
91         #captura o recebimento de um pacote TCP RST (reset).
92         print("\nAviso: o servidor foi desconectado inesperadamente.")
93     except Exception as e:
94         print(f"\nErro inesperado no cliente: {e}")
95     finally:
96         state.connection_closed = True
97         print("Encerrando conexão...")
98         sys.exit(0)
99
100 def main():
101     args = parse_args()
102     player_name = input("Digite seu nome de jogador: ")
103     state = LocalGameState()
104     #criação do socket:
105     #AF_INET: define a família de endereçamento como IPv4 (camada de rede).
106     #SOCK_STREAM: define o uso do protocolo TCP.
107     try:
108         #inicia o "3-way handshake" do TCP (SYN, SYN-ACK, ACK) com o servidor.
109         client_socket = socket.create_connection(
110             (args.host, args.port),
111             timeout=args.timeout,
112         )
113         client_socket.settimeout(None)
114     except ConnectionRefusedError:
115         print(f"Não foi possível conectar. O servidor está rodando em {args.host}:{args.port}?")
116         return
117     except socket.timeout:
118         print(f"Tempo esgotado ao conectar em {args.host}:{args.port}.")
119         print("Confira IP, porta, firewall e se ambos estão na mesma LAN/VPN.")
120         return
121     except socket.gaierror:
122         print(f"Endereço inválido ou não resolvido: {args.host}")
123         return
124     except OSError as exc:
125         print(f"Falha ao conectar em {args.host}:{args.port}: {exc}")
126         print("Confira IP, porta, firewall e se ambos estão na mesma LAN/VPN.")
127         return
128
129     #construção da PDU (protocol data unit) da camada de aplicação.
130     try:
131         send_msg(client_socket, "JOIN", player_name)

```

```

132 except OSError:
133     print("Não foi possível enviar JOIN. A conexão foi encerrada.")
134     client_socket.close()
135     return
136     #cria uma thread separada para lidar com a recepção bloqueante do socket
137     recv_thread = threading.Thread(target=recv_loop, args=(client_socket, state),
138     daemon=True)
138     recv_thread.start()
139
140     #Loop principal (thread principal) focado apenas em I/O do usuário e envio (upload
141     )
142     try:
143         while True:
144             if state.connection_closed:
145                 break
146
147             user_input = input("> ").strip()
148
149             if state.phase == "ENDED":
150                 if not user_input:
151                     send_msg(client_socket, "READY", {"player_id": state.my_id})
152                     print("Confirmação enviada. Aguardando outro jogador...")
153                 else:
154                     print("Pressione ENTER para jogar novamente.")
155                     continue
156
157             if state.is_spectator:
158                 print("[Você é espectador]")
159                 continue
160
161             if not user_input:
162                 continue
163             if not user_input.isalpha():
164                 print("Aviso: apenas letras (A-Z) são permitidas.")
165                 continue
166
167             #Regras do protocolo de aplicação para definir o "type" do payload
168             if len(user_input) == 1:
169                 msg_type = "GUESS_LETTER"
170             else:
171                 msg_type = "GUESS_WORD"
172             #envia para o servidor.
173             send_msg(client_socket, msg_type, user_input.upper())
174         except KeyboardInterrupt:
175             print("\nSaindo do jogo...")
176         except (ConnectionResetError, BrokenPipeError, OSError):
177             print("\nConexão com o servidor encerrada.")

```

```

177     finally:
178         client_socket.close()
179
180 if __name__ == "__main__":
181     main()

```

Listing A.1 – Arquivo cliente/client.py

A.2 Estado local do cliente

```

1 import json
2
3 class LocalGameState:
4     def __init__(self):
5         self.my_id = None #Necessário para comparar quem errou ou quem foi eliminado
6         self.phase = "WAITING"
7         self.revealed = ""
8         self.category = ""
9         self.my_attempts = []
10        self.all_players = [] #Lista de dicionários com nome, tentativas restantes e
pontuação
11        self.is_spectator = False
12        self.status_message = ""
13        self.connection_closed = False
14
15    def update(self, msg):
16        #Atualiza o estado local do jogo com base na mensagem recebida.
17        try:
18            #Garante que a mensagem seja um dicionário
19            if isinstance(msg, str):
20                data = json.loads(msg)
21            else:
22                data = msg
23
24            msg_type = data.get("type")
25            payload = data.get("payload", {})
26
27            #Dispatcher: mapeia o tipo de mensagem para o método privado
correspondente
28            dispatch_map = {
29                "WELCOME": self._on_welcome,
30                "GAME_START": self._on_game_start,
31                "GAME_OVER": self._on_game_over,
32                "STATE_UPDATE": self._on_state_update,
33                "WRONG_GUESS": self._on_wrong_guess,
34                "PLAYER_OUT": self._on_player_out,
35            }

```

```

36
37         if msg_type in dispatch_map:
38             dispatch_map[msg_type](payload)
39         else:
40             #Ignora silenciosamente ou loga tipos de mensagens não mapeados
41             pass
42
43     except json.JSONDecodeError:
44         print("\nErro ao decodificar a mensagem JSON do servidor.")
45     except Exception as e:
46         print(f"\nErro ao processar o estado local: {e}")
47
48     #--- Metodos de tratamento de estado ---
49     def _on_welcome(self, payload):
50         self.my_id = payload.get("your_id", self.my_id)
51
52     def _on_game_start(self, payload):
53         self.phase = "PLAYING"
54         self.my_attempts = []
55         self.category = payload.get("category", self.category)
56         self.status_message = ""
57
58         active_player_ids = payload.get("active_player_ids")
59         if active_player_ids:
60             self.is_spectator = self.my_id not in active_player_ids
61
62     def _on_game_over(self, payload):
63         self.phase = "ENDED"
64         self.status_message = ""
65
66     def _on_state_update(self, payload):
67         self.phase = payload.get("phase", self.phase)
68         self.revealed = payload.get("revealed", self.revealed)
69         self.all_players = payload.get("all_players", self.all_players)
70         self.status_message = payload.get("status_message", "")
71
72         for player in self.all_players:
73             if player.get("id") == self.my_id:
74                 self.is_spectator = not player.get("active", True)
75                 break
76
77     def _on_wrong_guess(self, payload):
78         #Atualiza my_attempts apenas se o erro foi do próprio jogador
79         if payload.get("player_id") == self.my_id:
80             guess = payload.get("guess")
81             if guess and guess not in self.my_attempts:
82                 self.my_attempts.append(guess)

```

```

83
84     def _on_player_out(self, payload):
85
86         eliminated_id = payload.get("player_id")
87         #Define como espectador se o eliminado for o próprio jogador
88         if eliminated_id== self.my_id:
89             self.is_spectator = True
90
91         #Permite que o renderer exiba o estado atualizado sem esperar pelo próximo
92         STATE_UPDATE.
93         for player in self.all_players:
94             if player.get("id") == eliminated_id:
95                 player["active"] = False
96                 break

```

Listing A.2 – Arquivo cliente/local_state.py

A.3 Renderização da interface

```

1  import os
2  from pathlib import Path
3
4  #Carregamento dos estágios da forca
5  def _load_gallows(path: Path) -> list[str]:
6      content = path.read_text(encoding="utf-8")
7      stages, current = [], []
8      for line in content.splitlines():
9          if line.strip() == "---":
10             stages.append("\n".join(current).strip())
11             current = []
12          else:
13             current.append(line)
14      stages.append("\n".join(current).strip())
15      if len(stages) != 7:
16          raise ValueError(f"Esperado 7 estágios na forca, encontrado {len(stages)}.")
17      return stages
18
19  _GALLOWS_PATH = next(
20      p for p in [
21          Path(__file__).resolve().parent / "forca.txt",
22          Path(__file__).resolve().parent.parent / "assets" / "forca.txt",
23      ]
24      if p.exists()
25  )
26  _GALLOWS: list[str] = _load_gallows(_GALLOWS_PATH)
27
28  #Utilitários

```

```

29 def clear_screen() -> None:
30     os.system("cls" if os.name == "nt" else "clear")
31
32
33 def _gallows_stage(wrong_count: int) -> str:
34     index = max(0, min(wrong_count, 6))
35     return _GALLOWS[index]
36
37
38 def _format_revealed(revealed: str) -> str:
39     if " " in revealed:
40         return revealed
41     return " ".join(revealed)
42
43 #Formata o placar lateral com nome, pontuação, tentativas e status
44 def _render_scoreboard(players: list[dict], my_id) -> str:
45     if not players:
46         return " (sem jogadores)"
47
48     lines = []
49     for p in players:
50         active = p.get("active", True)
51         is_me = p.get("id") == my_id
52
53         status = ""
54         if not active:
55             status = " [ESPECTADOR]"
56         elif is_me:
57             status = " você"
58
59         name = p.get("name", "?")
60         score = p.get("score", 0)
61         attempts = p.get("attempts_left", "?")
62
63         lines.append(f" {name}{status}")
64         lines.append(f" Pontos: {score} | Tentativas: {attempts}")
65
66     return "\n".join(lines)
67
68 #Funções públicas de renderização
69 def render_state(state) -> None:
70     clear_screen()
71
72     wrong_count = len(state.my_attempts)
73     gallows_text = _gallows_stage(wrong_count)
74     scoreboard = _render_scoreboard(state.all_players, state.my_id)
75

```

```

76     gallows_lines = gallows_text.splitlines()
77     board_lines   = [" PLACAR ", scoreboard]
78
79     print(gallows_text)
80     print()
81
82     print(" PLACAR ")
83     print(scoreboard)
84     print("")
85     print()
86
87     categoria = getattr(state, "category", "Desconhecida")
88     revealed = _format_revealed(state.revealed) if state.revealed else "?"
89
90     print(f" Categoria: {categoria}")
91     print(f" Palavra: {revealed}")
92     status_message = getattr(state, "status_message", "")
93     if status_message:
94         print(f" {status_message}")
95     print()
96
97     if state.my_attempts:
98         tentativas = " ".join(state.my_attempts)
99         print(f" Letras erradas: {tentativas}")
100     else:
101         print(" Letras erradas: (nenhuma ainda)")
102     print()
103
104     if state.is_spectator:
105         print(" Você é espectador. Acompanhe a partida.")
106     else:
107         print(" Sua vez digite uma letra ou a palavra completa:")
108
109
110 def render_waiting(connected: int, needed: int) -> None:
111     clear_screen()
112     print("")
113     print("          JOGO DA FORCA   Aguarde          ")
114     print("")
115     print(f" Jogadores conectados: {connected}/{needed}          ")
116     print(" Aguardando os demais jogadores...  ")
117     print("")
118
119
120 def render_game_over(winner_name: str, word: str, scores: list[dict]) -> None:
121     clear_screen()
122     print("")

```



```

123     print("                FIM DE JOGO                ")
124     print("")
125
126     if winner_name:
127         print(f"   Vencedor: {winner_name:<24}")
128     else:
129         print("   Nenhum vencedor desta vez.                ")
130
131     print(f"   Palavra: {word:<29}")
132     print("")
133     print("   PLACAR FINAL                ")
134
135     for p in sorted(scores, key=lambda x: x.get("score", 0), reverse=True):
136         name = p.get("name", "?")[:18]
137         score = p.get("score", 0)
138         print(f"        {name:<18} {score:>4} pts                ")
139
140     print("")
141     print()
142     print("Pressione ENTER para jogar novamente")

```

Listing A.3 – Arquivo interface/renderer.py

A.4 Servidor

```

1  import socket
2  import sys
3  import threading
4  import argparse
5  import os
6  from pathlib import Path
7  import time # Necessário para os delays
8
9  PROJECT_ROOT = Path(__file__).resolve().parents[1]
10 if str(PROJECT_ROOT) not in sys.path:
11     sys.path.insert(0, str(PROJECT_ROOT))
12
13 from utils.protocol import recv_msgs, send_msg
14 from servidor.game_state import ENDED, PLAYING, GameState
15 from servidor.word_manager import load_words, pick_word
16
17 DEFAULT_HOST = os.environ.get("HANGMAN_HOST", "0.0.0.0")
18 DEFAULT_PORT = int(os.environ.get("HANGMAN_PORT", "5000"))
19 MIN_PLAYERS = 2
20 MAX_CLIENTS = 3
21 WORD_SOLVED_DELAY_SECONDS = 2
22

```

```

23 WORDS_PATH = PROJECT_ROOT / "assets" / "palavras.txt"
24 available_words = []
25
26 #Estado global da partida compartilhado entre todas as threads.
27 game_state = GameState()
28 next_player_id = 1
29 next_player_id_lock = threading.Lock()
30 ready_player_ids: set[int] = set()
31 ready_lock = threading.Lock()
32 last_game_over_payload = None
33
34 def _broadcast_state(status_message: str | None = None) -> None:
35     game_state.broadcast("STATE_UPDATE", game_state.get_state_payload(status_message))
36
37
38 def _pop_next_word() -> tuple[str, str] | None:
39     if not available_words:
40         return None
41
42     word_tuple = pick_word(available_words)
43     available_words.remove(word_tuple)
44     return word_tuple
45
46
47 def _start_next_round(
48     reset_scores: bool,
49     active_player_ids: set[int] | None = None,
50 ) -> bool:
51     word_tuple = _pop_next_word()
52     if word_tuple is None:
53         return False
54
55     word, category = word_tuple
56     if reset_scores and active_player_ids is None:
57         active_player_ids = game_state.connected_player_ids()
58
59     game_state.start_game(
60         word,
61         category,
62         reset_scores=reset_scores,
63         active_player_ids=active_player_ids,
64     )
65     game_state.broadcast("GAME_START", {
66         "category": category,
67         "word_length": len(word),
68         "active_player_ids": list(game_state.active_player_ids()),
69     })

```

```

70     _broadcast_state()
71     print(f"[GAME] Round iniciado! Palavra: {word} ({category})")
72     return True
73
74
75 def _broadcast_game_over(winner_override: dict | None = None) -> None:
76     global last_game_over_payload
77
78     game_state.mark_ended()
79     winner = winner_override or game_state.determine_winner()
80     payload = {
81         "winner_id": winner["id"] if winner else None,
82         "winner_name": winner["name"] if winner else None,
83         "word": game_state.current_word(),
84         "scores": game_state.get_final_scores(),
85     }
86     last_game_over_payload = payload
87     with ready_lock:
88         ready_player_ids.clear()
89     game_state.broadcast("GAME_OVER", payload)
90
91
92 def _handle_word_solved() -> None:
93     _broadcast_state("Palavra descoberta! Trocando palavra...")
94     time.sleep(WORD_SOLVED_DELAY_SECONDS)
95
96     if game_state.current_phase() != PLAYING:
97         return
98
99     if _start_next_round(reset_scores=False):
100         return
101
102     print(f"[GAME] Banco de palavras esgotado.")
103     _broadcast_game_over()
104
105
106 def _handle_ready(player_id: int) -> None:
107     global available_words
108
109     if game_state.current_phase() != ENDED:
110         return
111
112     connected_ids = game_state.connected_player_ids()
113     if player_id not in connected_ids:
114         return
115
116     with ready_lock:

```

```

117     ready_player_ids.add(player_id)
118     ready_connected_ids = ready_player_ids & connected_ids
119     if len(ready_connected_ids) < MIN_PLAYERS:
120         return
121     starting_player_ids = set(ready_connected_ids)
122     ready_player_ids.clear()
123
124     available_words = load_words(WORDS_PATH)
125     if not _start_next_round(
126         reset_scores=True,
127         active_player_ids=starting_player_ids,
128     ):
129         game_state.broadcast("ERROR", {"message": "Banco de palavras vazio."})
130
131 def _handle_guess(player_id: int, letter: str) -> None:
132     result = game_state.process_guess(player_id, letter)
133
134     if not result["valid"]:
135         return
136
137     if result["correct"]:
138         game_state.broadcast("CORRECT_GUESS", {
139             "player_id": player_id,
140             "letter": letter.upper(),
141             "positions": result["positions"],
142         })
143     else:
144         game_state.broadcast("WRONG_GUESS", {
145             "player_id": player_id,
146             "guess": letter.upper(),
147         })
148
149     #Jogador zerou tentativas espectador.
150     if result["eliminated"]:
151         game_state.broadcast("PLAYER_OUT", {"player_id": player_id})
152
153     #Alguém acertou a palavra.
154     if result["won"]:
155         _handle_word_solved()
156         return
157
158     #Todos viraram espectadores (sem vencedor por acerto).
159     if not game_state.active_player_ids():
160         _broadcast_game_over()
161         return
162
163     _broadcast_state()

```

```

164
165
166 def _handle_word_guess(player_id: int, word: str) -> None:
167     result = game_state.process_word_guess(player_id, word)
168
169     if not result["valid"]:
170         return
171
172     if result["correct"]:
173         game_state.broadcast("CORRECT_GUESS", {
174             "player_id": player_id,
175             "word": word.upper(),
176             "positions": result["positions"],
177         })
178         _handle_word_solved()
179         return
180
181     game_state.broadcast("WRONG_GUESS", {
182         "player_id": player_id,
183         "guess": word.upper(),
184     })
185
186     if result["eliminated"]:
187         game_state.broadcast("PLAYER_OUT", {"player_id": player_id})
188
189     if not game_state.active_player_ids():
190         _broadcast_game_over()
191         return
192
193     _broadcast_state()
194
195 def handle_client(conn: socket.socket, addr, player_id: int) -> None:
196     """Atende um cliente em thread dedicada."""
197     global available_words
198
199     recv_buffer = [""]
200
201     try:
202         while True:
203             messages = recv_msgs(conn, recv_buffer)
204             if not messages:
205                 break
206
207             for msg in messages:
208                 msg_type = msg.get("type")
209                 payload = msg.get("payload")
210

```

```

211         if msg_type == "JOIN":
212             player_name = payload if isinstance(payload, str) else str(payload
213     )
214
215             game_state.add_player(player_id, player_name, sock=conn)
216
217             print(f"[JOIN] Jogador {player_id} ({player_name}) entrou")
218
219             connected = game_state.connected_count()
220             send_msg(conn, "WELCOME", {
221                 "your_id": player_id,
222                 "player_count": connected,
223             })
224
225             if game_state.current_phase() == PLAYING:
226                 send_msg(conn, "GAME_START", {
227                     "category": game_state.category,
228                     "word_length": len(game_state.word),
229                     "active_player_ids": list(game_state.active_player_ids()),
230                 })
231                 _broadcast_state()
232                 continue
233
234             if game_state.current_phase() == ENDED:
235                 if last_game_over_payload is not None:
236                     send_msg(conn, "GAME_OVER", last_game_over_payload)
237                     continue
238
239             #Mantém a sala em espera somente antes de a partida começar.
240             if connected < MIN_PLAYERS:
241                 game_state.broadcast("WAITING", {
242                     "connected": connected,
243                     "needed": MIN_PLAYERS,
244                 })
245                 continue
246
247             #Segundo jogador conectado inicia a partida.
248             available_words = load_words(WORDS_PATH)
249             if not _start_next_round(reset_scores=True):
250                 send_msg(conn, "ERROR", {"message": "Banco de palavras vazio."
251     })
252
253     elif msg_type == "GUESS_LETTER" and game_state.current_phase() ==
254     PLAYING:
255         letter = payload if isinstance(payload, str) else ""
256         _handle_guess(player_id, letter)

```

```

254         elif msg_type == "GUESS_WORD" and game_state.current_phase() ==
PLAYING:
255             word = payload if isinstance(payload, str) else ""
256             _handle_word_guess(player_id, word)
257
258         elif msg_type == "READY":
259             _handle_ready(player_id)
260
261     except (ConnectionResetError, OSError):
262         #Desconexão abrupta (TCP RST ou terminal fechado).
263         pass
264
265     finally:
266         print(f"[-] Jogador {player_id} desconectado")
267
268         #remove_player marca o jogador como espectador e verifica se
269         #sobrou apenas 1 ativo nesse caso retorna trigger_game_over.
270         result = game_state.remove_player(player_id)
271         game_state.broadcast("PLAYER_OUT", {"player_id": player_id})
272
273         if result and result.get("trigger_game_over"):
274             print(f"[GAME] Jogador {result['winner_name']} venceu por W.O.")
275             _broadcast_game_over({
276                 "id": result["winner_id"],
277                 "name": result["winner_name"],
278             })
279         elif game_state.current_phase() == PLAYING:
280             _broadcast_state()
281
282         try:
283             conn.close()
284         except OSError:
285             pass
286
287 def accept_loop(server_sock: socket.socket) -> None:
288     global next_player_id
289
290     while True:
291         conn, addr = server_sock.accept()
292
293         with next_player_id_lock:
294             if game_state.connected_count() >= MAX_CLIENTS:
295                 send_msg(conn, "ERROR", {"message": "Servidor lotado. Limite de 3
jogadores."})
296                 conn.close()
297                 continue
298

```

```

299         player_id      = next_player_id
300         next_player_id += 1
301
302         print(f"[+] Jogador {player_id} conectado de {addr}")
303         client_thread = threading.Thread(
304             target=handle_client,
305             args=(conn, addr, player_id),
306             daemon=True,
307         )
308         client_thread.start()
309
310
311 def local_ipv4_addresses() -> list[str]:
312     addresses = set()
313     try:
314         hostname = socket.gethostname()
315         for info in socket.getaddrinfo(hostname, None, socket.AF_INET):
316             ip = info[4][0]
317             if not ip.startswith("127."):
318                 addresses.add(ip)
319     except OSError:
320         pass
321
322     return sorted(addresses)
323
324
325 def parse_args() -> argparse.Namespace:
326     parser = argparse.ArgumentParser(description="Servidor do jogo da forca
327     competitivo.")
328     parser.add_argument(
329         "--host",
330         default=DEFAULT_HOST,
331         help="Interface para escutar conexões. Use 0.0.0.0 para LAN.",
332     )
333     parser.add_argument(
334         "--port",
335         type=int,
336         default=DEFAULT_PORT,
337         help="Porta TCP do servidor.",
338     )
339     return parser.parse_args()
340
341 def main() -> None:
342     args = parse_args()
343     server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
344     server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

```



```

345     server_sock.bind((args.host, args.port))
346     server_sock.listen()
347     print(f"Servidor escutando em {args.host}:{args.port}")
348     if args.host == "0.0.0.0":
349         ips = local_ipv4_addresses()
350         if ips:
351             print("Clientes na LAN/VPN devem conectar usando um destes IPs:")
352             for ip in ips:
353                 print(f"  python3 cliente/client.py {ip} --port {args.port}")
354         else:
355             print("Use o IP local da máquina servidora no cliente, não 0.0.0.0.")
356
357     try:
358         accept_loop(server_sock)
359     except KeyboardInterrupt:
360         print("\nServidor encerrado manualmente.")
361     finally:
362         server_sock.close()
363
364
365 if __name__ == "__main__":
366     main()

```

Listing A.4 – Arquivo servidor/game_server.py

A.5 Estado global do jogo

```

1  from __future__ import annotations
2
3  import socket
4  import threading
5  import unicodedata
6  from dataclasses import dataclass, field
7  from typing import Any
8
9  from utils.protocol import send_msg
10
11
12  WAITING = "WAITING"
13  PLAYING = "PLAYING"
14  ENDED = "ENDED"
15  DEFAULT_ATTEMPTS = 6
16
17
18  @dataclass
19  class PlayerData:
20      player_id: int

```

```

21     name: str
22     attempts: int = DEFAULT_ATTEMPTS
23     score: int = 0
24     is_spectator: bool = False
25     correct_unique_letters: set[str] = field(default_factory=set)
26
27
28 class GameState:
29     def __init__(self, word: str = "", category: str = "") -> None:
30         self.phase = WAITING
31         self.word = self._normalize_word(word)
32         self.category = category
33         self.revealed = ["_" if char.isalpha() else char for char in self.word]
34         self.guessed_letters: set[str] = set()
35         self.players: dict[int, PlayerData] = {}
36         self.sockets: dict[int, socket.socket] = {}
37         self.lock = threading.Lock()
38
39     def add_player(
40         self,
41         player_id: int,
42         name: str,
43         sock: socket.socket | None = None,
44     ) -> PlayerData:
45         with self.lock:
46             player = self.players.get(player_id)
47             if player is None:
48                 player = PlayerData(player_id=player_id, name=name)
49                 self.players[player_id] = player
50
51             if sock is not None:
52                 self.sockets[player_id] = sock
53
54             return player
55
56     def remove_player(self, player_id: int) -> dict[str, Any] | None:
57         with self.lock:
58             self.sockets.pop(player_id, None)
59             player = self.players.get(player_id)
60             if player is None:
61                 return None
62
63             player.is_spectator = True
64
65             if self.phase != PLAYING:
66                 return None
67

```

```

68         active = [p for p in self.players.values() if not p.is_spectator]
69         if len(active) == 1:
70             winner = active[0]
71             self.phase = ENDED
72             return {
73                 "trigger_game_over": True,
74                 "winner_id": winner.player_id,
75                 "winner_name": winner.name,
76             }
77
78         return None
79
80     def start_game(
81         self,
82         word: str,
83         category: str,
84         reset_scores: bool = True,
85         active_player_ids: set[int] | None = None,
86     ) -> None:
87         with self.lock:
88             self.phase = PLAYING
89             self.word = self._normalize_word(word)
90             self.category = category
91             self.revealed = ["_" if char.isalpha() else char for char in self.word]
92             self.guessed_letters = set()
93             for player in self.players.values():
94                 if reset_scores:
95                     player.score = 0
96                     player.is_spectator = (
97                         active_player_ids is not None
98                         and player.player_id not in active_player_ids
99                     )
100
101                 if not player.is_spectator:
102                     player.attempts = DEFAULT_ATTEMPTS
103
104                     player.correct_unique_letters = set()
105
106     def reset(self) -> None:
107         with self.lock:
108             self.phase = WAITING
109             self.word = ""
110             self.category = ""
111             self.revealed = []
112             self.guessed_letters = set()
113             for player in self.players.values():
114                 player.score = 0

```

```

115         player.attempts = DEFAULT_ATTEMPTS
116         player.is_spectator = False
117         player.correct_unique_letters = set()
118
119     def mark_ended(self) -> None:
120         with self.lock:
121             self.phase = ENDED
122
123     def current_phase(self) -> str:
124         with self.lock:
125             return self.phase
126
127     def connected_count(self) -> int:
128         with self.lock:
129             return len(self.sockets)
130
131     def connected_player_ids(self) -> set[int]:
132         with self.lock:
133             return set(self.sockets)
134
135     def active_player_ids(self) -> set[int]:
136         with self.lock:
137             return {p.player_id for p in self.players.values() if not p.is_spectator}
138
139     def process_guess(self, player_id: int, letter: str) -> dict[str, Any]:
140         with self.lock:
141             player = self.players.get(player_id)
142             if player is None:
143                 return self._result(valid=False)
144             if player.is_spectator:
145                 return self._result(valid=False, eliminated=True)
146
147             normalized_letter = self._normalize_letter(letter)
148             if normalized_letter is None:
149                 return self._result(valid=False)
150
151             if normalized_letter in self.guessed_letters:
152                 return self._result(valid=False)
153
154             self.guessed_letters.add(normalized_letter)
155             positions = [
156                 index
157                 for index, char in enumerate(self.word)
158                 if char == normalized_letter
159             ]
160             correct = bool(positions)
161

```

```

162         if correct:
163             for index in positions:
164                 self.revealed[index] = normalized_letter
165                 player.score += len(positions)
166                 player.correct_unique_letters.add(normalized_letter)
167         else:
168             self._decrement_attempt(player)
169
170         return self._result(
171             valid=True,
172             correct=correct,
173             won=self._is_won(),
174             eliminated=player.is_spectator,
175             positions=positions,
176         )
177
178     def process_word_guess(self, player_id: int, word: str) -> dict[str, Any]:
179         with self.lock:
180             player = self.players.get(player_id)
181             if player is None:
182                 return self._result(valid=False)
183             if player.is_spectator:
184                 return self._result(valid=False, eliminated=True)
185
186             normalized_word = self._normalize_word_guess(word)
187             if normalized_word is None:
188                 return self._result(valid=False)
189
190             correct = normalized_word == self.word
191             positions: list[int] = []
192             if correct:
193                 self.revealed = list(self.word)
194                 player.score += len({char for char in self.word if char.isalpha()})
195                 player.correct_unique_letters.update(
196                     char for char in self.word if char.isalpha()
197                 )
198                 positions = [
199                     index
200                     for index, char in enumerate(self.word)
201                     if char.isalpha()
202                 ]
203             else:
204                 self._decrement_attempt(player)
205
206             return self._result(
207                 valid=True,
208                 correct=correct,

```

```

209         won=self._is_won(),
210         eliminated=player.is_spectator,
211         positions=positions,
212     )
213
214     def broadcast(self, msg_type: str, data: Any) -> None:
215         with self.lock:
216             sockets_snapshot = list(self.sockets.items())
217             snapshot_by_player = dict(sockets_snapshot)
218
219             failed_player_ids: list[int] = []
220             for player_id, sock in sockets_snapshot:
221                 try:
222                     send_msg(sock, msg_type, data)
223                 except OSError:
224                     failed_player_ids.append(player_id)
225
226             if failed_player_ids:
227                 with self.lock:
228                     for player_id in failed_player_ids:
229                         if self.sockets.get(player_id) is snapshot_by_player.get(player_id
230 ):
231                             self.sockets.pop(player_id, None)
232
233     def get_state_payload(self, status_message: str | None = None) -> dict[str, Any]:
234         with self.lock:
235             payload = {
236                 "phase": self.phase,
237                 "revealed": " ".join(self.revealed),
238                 "all_players": [
239                     {
240                         "id": p.player_id,
241                         "name": p.name,
242                         "attempts_left": p.attempts,
243                         "score": p.score,
244                         "active": not p.is_spectator,
245                     }
246                     for p in self.players.values()
247                 ],
248             }
249             if status_message:
250                 payload["status_message"] = status_message
251             return payload
252
253     def current_word(self) -> str:
254         with self.lock:
255             return self.word

```

```

255
256 def get_final_scores(self) -> list[dict[str, Any]]:
257     with self.lock:
258         return sorted(
259             [
260                 {
261                     "id": p.player_id,
262                     "name": p.name,
263                     "score": p.score,
264                     "unique_letters": len(p.correct_unique_letters),
265                 }
266                 for p in self.players.values()
267             ],
268             key=lambda x: (x["score"], x["unique_letters"]),
269             reverse=True,
270         )
271
272 def determine_winner(self) -> dict[str, Any] | None:
273     scores = self.get_final_scores()
274     if not scores:
275         return None
276     return scores[0]
277
278 def _decrement_attempt(self, player: PlayerData) -> None:
279     player.attempts = max(0, player.attempts - 1)
280     if player.attempts == 0:
281         player.is_spectator = True
282
283 def _is_won(self) -> bool:
284     return bool(self.word) and "".join(self.revealed) == self.word
285
286 def _result(
287     self,
288     *,
289     valid: bool,
290     correct: bool = False,
291     won: bool = False,
292     eliminated: bool = False,
293     positions: list[int] | None = None,
294 ) -> dict[str, Any]:
295     return {
296         "valid": valid,
297         "correct": correct,
298         "won": won,
299         "eliminated": eliminated,
300         "positions": positions or [],
301     }

```

```

302
303 def _normalize_letter(self, letter: str) -> str | None:
304     if not isinstance(letter, str):
305         return None
306     normalized = self._remove_accents(letter.strip()).upper()
307     if len(normalized) != 1 or not normalized.isalpha():
308         return None
309     return normalized
310
311 def _normalize_word_guess(self, word: str) -> str | None:
312     if not isinstance(word, str):
313         return None
314     normalized = self._normalize_word(word.strip())
315     if not normalized or not normalized.replace(" ", "").isalpha():
316         return None
317     return normalized
318
319 def _normalize_word(self, word: str) -> str:
320     return self._remove_accents(word).upper()
321
322 def _remove_accents(self, value: str) -> str:
323     normalized = unicodedata.normalize("NFD", value)
324     return "".join(char for char in normalized if unicodedata.category(char) != "
Mn")

```

Listing A.5 – Arquivo servidor/game_state.py

A.6 Gerenciador de palavras

```

1  """Carregamento e sorteio de palavras do banco estatico do jogo."""
2
3  from __future__ import annotations
4
5  import random
6  from pathlib import Path
7
8
9  def load_words(path: str | Path) -> list[tuple[str, str]]:
10     """Le palavras.txt e retorna lista de tuplas (palavra, categoria)."""
11     words: list[tuple[str, str]] = []
12     file_path = Path(path)
13
14     with file_path.open(encoding="utf-8") as file:
15         for line in file:
16             line = line.strip()
17             if not line:
18                 continue

```



```

19
20         word, category = line.split(",", maxsplit=1)
21         words.append((word.strip(), category.strip()))
22
23     return words
24
25
26 def pick_word(words: list[tuple[str, str]]) -> tuple[str, str]:
27     """Sorteia uma palavra aleatoriamente do banco."""
28     return random.choice(words)

```

Listing A.6 – Arquivo servidor/word_manager.py

A.7 Protocolo de comunicação

```

1  """Protocol framing helpers for newline-delimited JSON messages.
2
3  Message types catalog:
4  JOIN, GUESS_LETTER, GUESS_WORD, WELCOME, WAITING, GAME_START,
5  STATE_UPDATE, WRONG_GUESS, CORRECT_GUESS, PLAYER_OUT, GAME_OVER, READY, ERROR.
6  """
7
8  from __future__ import annotations
9
10 import json
11 import socket
12 from typing import Any
13
14
15 def send_msg(sock: socket.socket, msg_type: str, data: Any) -> None:
16     """Serialize and send one protocol message."""
17     payload = {"type": msg_type, "payload": data}
18     encoded = (json.dumps(payload, ensure_ascii=False) + "\n").encode("utf-8")
19     sock.sendall(encoded)
20
21
22 def recv_msgs(sock: socket.socket, buffer: list[str]) -> list[dict[str, Any]]:
23     """Read from socket, accumulate fragments, and return complete messages."""
24     if not buffer:
25         buffer.append("")
26
27     chunk = sock.recv(4096)
28     if not chunk:
29         return []
30
31     buffer[0] += chunk.decode("utf-8")
32     parts = buffer[0].split("\n")

```

```
33     buffer[0] = parts.pop()
34
35     messages: list[dict[str, Any]] = []
36     for raw in parts:
37         line = raw.strip()
38         if not line:
39             continue
40         messages.append(json.loads(line))
41
42     return messages
```

Listing A.7 – Arquivo utils/protocol.py